# EE 332: Introduction to Computer Vision

# Offset Measurement Using Computer Vision

**By: Tomasz Krzeminski**
**Date: 12/10/2025**

# Table of Contents

# 1 Introduction

Computer vision as a field allows us to branch the physical world with the digital world. It enables us to transform the physical representation of objects into a projected 2D representation, which we call the image frame. In the analysis of this image frame, we can perform a series of operations to extract various forms of information about the observable world. The operations and techniques we have overviewed in this class can be categorized into low, medium and high level operations. The low level operations are focused on image extraction and preprocessing, which includes topics like histogram equalization, image blurring, morphological operations and connected component labeling (CCL). Medium level operations are then focused on things like feature extraction and segmentation, which overviews topics like texture extraction, segmentation, edge detection and curve fitting via the Hough Transform. Lastly, the high level operations we discussed are focused on detection, tracking and 3D reconstruction, in which topics like face detection, motion tracking, camera pose reconstruction and feature extraction using methods like SIFT, were covered.

The topics overviewed above can be utilized in tandem with one another to shape a project with higher level functionality. For example, image segmentation has been used in medical image analysis to identify various features of microbial life. In order to get a viable segmentation, however, various image processing techniques, along with feature extraction methods, are employed to construct an image that is viable for segmentation. This is a simple yet powerful application of computer vision, which serves as a motivation for continued research on how it can be applied in various fields. My interests lie in the field of robotics, and as such I would like to learn more about 3D reconstruction and motion tracking, which are necessary in applications like SLAM for mobile robots. Furthermore, it is important that the images and

complimentary information are clear and easy to extract. This is a necessity as the internal state representation of a mobile robot is heavily reliant on its measurement data. As such, being able to extract meaningful features from images quickly is a topic that I am interested in. However, for the scope of this project, my research will focus on effective and reliable geometric feature extraction which can be used by users to understand the geometric properties of an object and its internal features. The motivations for this project are discussed in section 2.

## 2 Project Description

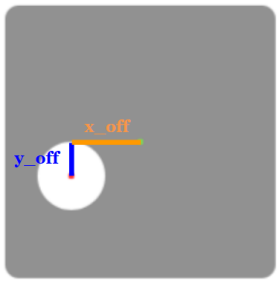


**Figure 1: Example Output of an Offset Measurement on a Test Image**

Feature accuracy is a topic of interest for product automation in industry. As such, manufacturing plants require that machine operators constantly check whether the features are within an acceptable tolerance for proper functionality. This process often requires that the operators pause their production lines, inspect the product and the internal features for misalignment or defects, and adjust the parameters to correct for these errors. The inspection process often requires that the operators have to measure each feature by hand, which takes time and can introduce human error. Whenever the production line is restocked or changed over, the operators have to perform this configuration process. This is wasteful and inefficient as the configuration process can take up to half an hour if the configuration is particularly difficult, which ultimately results in prolonged production downtimes and, therefore, product loss. In

order to improve the configuration time and measurement accuracy, a quick, accurate and easy to use measurement software can be developed.

This project will overview an offset measurement software which utilizes computer vision to measure geometric properties of objects, as well as its internal features, and will measure offsets between the centroid of the object and each internal feature. The major constraint of this project is that the measurements will only be performed on one face of the object, meaning that no depth recovery is necessary. As such, the goal of this project is to develop a tool that is capable of measuring geometric properties, such as diameters for circular objects and widths and heights for objects that can be approximated by a rectangular bounding box. Furthermore, the centroids of the object and any internal feature (i.e. holes, engravings, etc.) should also be able to be measured as well. With these general geometric properties determined, the offset measurements between the object centroid and each internal feature centroid will need to be measured as is seen in Figure 1. Lastly, because this is a measurement tool, the resulting output and user interface should be intuitive and easy to read for ease of use. Additionally, the measurement accuracy should be within an acceptable range for viable measurements.

## 3 Design Approach

To implement the features aforementioned above, I followed a two stage implementation process. Stage 1 focused on developing rudimentary software that will work on simple test images made in a software like paint. Furthermore, because the test images could be fully defined, the results should be fully deterministic and accurate as there is no noise from things like lighting difference or camera distortion. This means that the measurements of the size and offsets should have little to no variation from the real image. In addition to this, it is assumed that

the object of interest in each image has zero rotation, meaning that the offset measurements can be calculated in the image frame.

Stage 2 focuses on modifying the existing implementation to apply it to real images, or a video stream from a webcam. Real images introduce a number of challenges, some of which include lighting variation, lens distortion, 3D to 2D project and camera pose misalignment. These challenges can cause the edge detection and final measurements to be more noisy, and therefore more uncertain. In addition to this, in order to obtain physical measurements in units of inches or millimeters, a pixel-to-inch or pixel-to-millimeter correspondence needs to be determined. All of these things can greatly impact the effectiveness of the measurements, and therefore require special considerations in their design. Lastly, the placement of the object under the camera requires the user to align it themselves. As such, it is often difficult to get a perfect alignment, which means that the object has some level of rotation. The challenges caused by object rotation will be discussed in section 3.2.3.

### 3.1 Stage 1

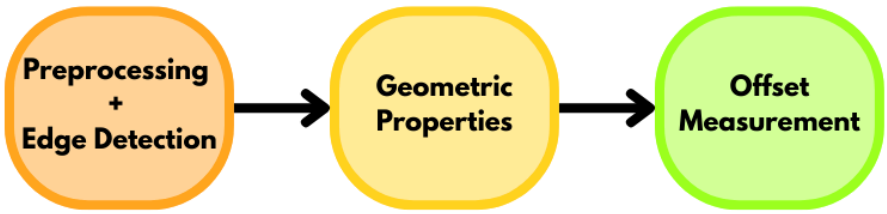


**Figure 2: Implementation Pipeline for Stage 1**

Figure 2 overviews the implementation pipeline followed to achieve the aforementioned goals for Stage 1. The process starts with the preprocessing and edge detection of the objects and internal features. This step focuses on cleaning up the images so that the thresholds for edge detection are well defined and so that the edge detection creates connected edges viable for

contour extraction. Once the contours have been defined, the process goes on to extract the geometric properties. In this step, geometric properties like diameter, width, height and the centroids of each object and feature are determined. Lastly, the centroids found in the previous step are then analyzed to determine all the offset measurements between the body centroid and each internal feature centroid. The following sections will overview the pipeline in greater detail.

### 3.1.1 Preprocessing and Edge Detection

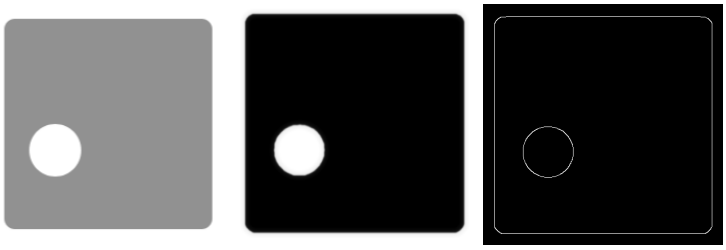

**Figure 3: Input Image (left), Preprocessed Image (middle) and Canny Edge Detection + Closing Operation (right)**

The preprocessing and edge detection step starts by taking in a RGB image provided by the user. This image is then equalized via Histogram Equalization, which aims to enhance the contrast in the images by adjusting the pixel intensities in the image to create a more uniformly distributed histogram. The equalized image is then blurred, first by applying a Gaussian Blur with a kernel of size 9x9 and a sigma (standard deviation) of 2. Because of the larger kernel and higher sigma value, the blurring is more intense, making the image smoother while reducing the presence of any noise present in the image. Because of the intense blurring caused by the Gaussian Blur, a Bilateral Filter is applied to preserve the edge contents of the objects, and reduce any remaining noise. According to Paris et. al. (2008, p.2), the Bilateral Filter works by replacing each pixel with the average value of all its neighbors. The combination of both of these blurs results in an image that has a lot of noisy information, such as texture or lighting variation, reduced while preserving the dominant edge contents such as those seen around the border of the

object and any internal feature. The preprocessing techniques discussed above are less noticeable in this stage, as is seen in the middle image of Figure 3, as the images are very simplistic and highly processed, which means less noise to filter out.

With the image blurred, the thresholds for Canny Edge Detection can be determined. To make the implementation more robust, an adaptive thresholding method is required as objects can have different colors or hues, which will result in different thresholds for each object. To employ this adaptive thresholding, Otsu's method can be used. The authors at OpenCV (n.d) describe Otsu's method as an algorithm that finds an optimal threshold by taking an image, which is assumed to be bimodal (two distinct image values), and extracting its histogram which has two distinct peaks. Otsu's method then finds an intensity value that lies in the middle of the two peaks, resulting in an ideal threshold for the image. Because the blurred image is a grayscale image, Otsu's method can be applied to determine an ideal threshold. The threshold obtained from Ostu's method is then treated as an upper threshold, and a lower threshold is calculated to be half the upper threshold. With both thresholds determined, Canny Edge Detection is performed to extract the edges of all the features. These edges then need to be used to create connected contours, and as such need to be processed to ensure that they are closed. To do this, the Closing morphological operation is done, employing a 5x5 rectangular structuring element. The extracted edges of the example input image can be seen in the right image of Figure 3.

### 3.1.2 Geometric Properties

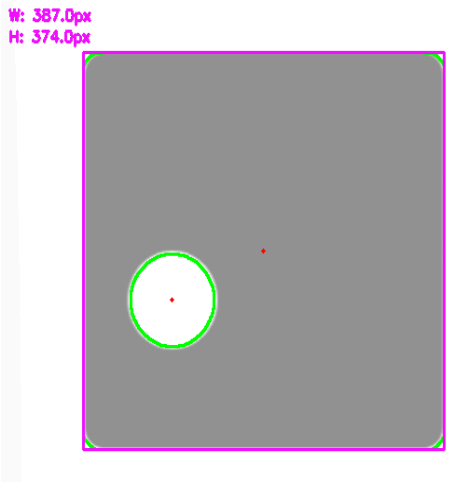


**Figure 4: Object Measurements and Centroid (Marked by Red Dot)**

The geometric properties step focuses on extracting the geometric properties of the objects as well as their centroids. To start, the centroids of the object and each internal feature must be computed.

$$\bar{x} = \frac{\int x\, dA}{\int dA} = \frac{Q_x}{A} \quad \textbf{(1)}$$

$$\bar{y} = \frac{\int y\, dA}{\int dA} = \frac{Q_y}{A} \quad \textbf{(2)}$$

Equations (1) and (2) describe centroid computation of any object, in which the First Moments of Area with respect to each coordinate are computed and then divided by the overall area of the object. These properties are easily determined by analyzing the contours found thanks to the edge detections in the previous step. OpenCV offers a simple function that extracts the necessary components, after which the centroids are computed. From Figure 4, it can be seen that the centroids of the object and its internal feature are accurately placed as indicated by the red dots.

With the centroids found, geometric properties like the diameter or width and height can be determined. In order to decide which geometric property to use, the circularity of the object

needs to be determined. To do this, an analysis of the ratio between the longest and shortest radius from the center can be done. For a circle, the ratio between longest and shortest radii should be roughly equal to 1.0. However, some circular objects have slight elongation, meaning that a threshold value of roughly 0.9 should suffice. For square or rectangular objects, the longest radius will always be longer than the shortest radius, resulting in a smaller ratio. Given this condition, the circularity of an object can be determined.

If an object is determined to be circular, a circular bounding box will be fitted around the contour of an object. From this bounding circle, the diameter can easily be determined. On the other hand, if the object is determined to be non-circular, a bounding box will be fitted to the contour. Even if the object is not rectangular, its primary dimensions can be captured by the bounding box. As such, the width and height of the bounding box can be used to determine the principal width and height of the object. All of the measurements done in this section are kept in units of pixels (px) as is seen in Figure 4.

### 3.1.3 Offset Measurement (No Rotation)

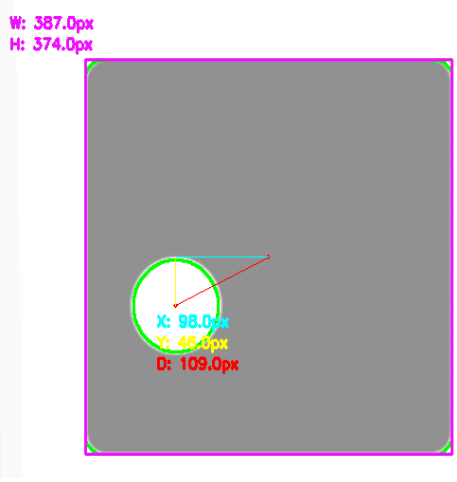


**Figure 5: Offset Measurements on a Simple Image**

Lastly, offset measurement begins by identifying the centroids of the object and each internal feature, which was done in the previous step. Following this, the contour hierarchy of each contour needs to be extracted to assign the outermost contour as the root contour, and each internal feature as the internal contours. This hierarchy can easily be determined by means of OpenCVs contour function, which keeps track of all the contours and their corresponding place in the hierarchy. An assumption made for Stage 1 is that the objects in each image are not rotated, meaning that the object frame axis is aligned with the image frame axis. This assumption means that the offset measurement can be performed in the image frame, and is computed thanks to equation (3) and (4).

$$dx = \bar{x}_r - \bar{x} \quad \textbf{(3)}$$

$$dy = \bar{y}_r - \bar{y} \quad \textbf{(4)}$$

Both equations are simply stating that taking the difference between the x and y values respectively of the root centroid ($\bar{x}_r$) and any internal feature centroid ($\bar{x}$). The result of performing the computation returns the length of the offset measurement vector in each dimension. Because of the contour hierarchy, each offset measurement will reference the same root contour as long as they are a part of the same object. Additionally, multiple offset measurements may be present per object, as long as they have an assigned centroid and are found to be in the contour hierarchy of the object. With the lengths of each offset vector determined, the plotting of the lines corresponding vector lines can be done by assembling an end point for both the y-offset and x-offset lines. This point is simply determined by adding the x-length obtained from equation (3) to the x-value of the root centroid for the x-position, and by subtracting the y-length from the y-value of the internal feature centroid for the y-position. The resultant point lies on the same y-position as the root centroid and is simply shifted by the

x-length, as is seen in Figure 5. In this figure you can see that the offset measurements are correct, and their corresponding measurement values are plotted near the internal feature centroid.

### 3.2 Stage 2

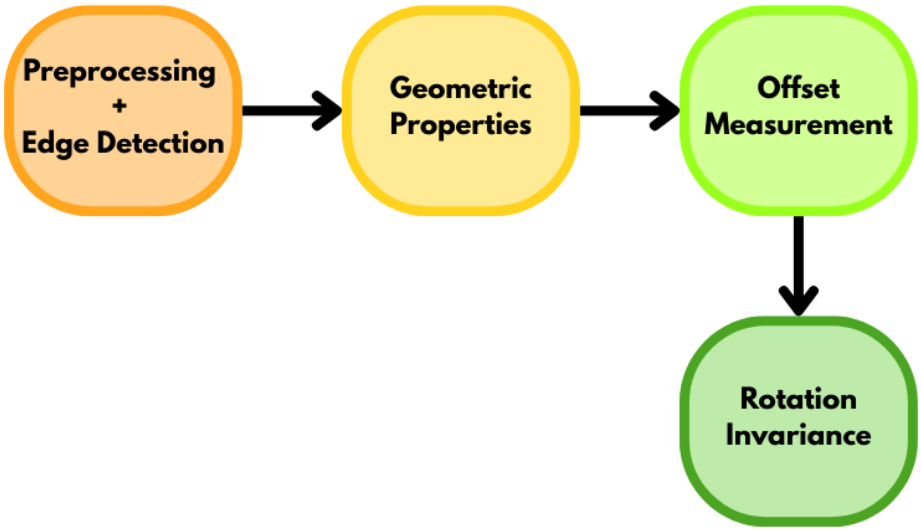


**Figure 6: Implementation Pipeline for Stage 2**

The implementation process, as is modeled by Figure 6, shows that it is exactly the same as in Stage 1, with the added step of implementing rotational invariance to the process. We again start with image processing and edge detection, which is largely the same, with some modification. The geometric properties section is also similar to Stage 1, with an additional step of the configuration for units of measurement. The offset measurement step is exactly the same, however, we explore the issue of assuming object rotation. The last step addresses object rotation and introduces rotational invariance to the implementation.

### 3.2.1 Preprocessing and Edge Detection

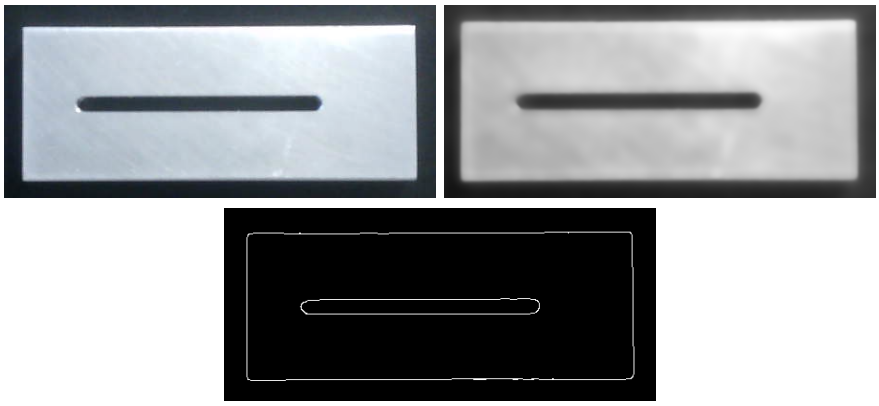

**Figure 7: Input Image (top-left), Preprocessed Image (top-right) and Canny Edge Detection + Closing Operation (bottom-middle)**

Similar to Stage 1, the first step in Stage 2 begins with preprocessing and edge detection. From Figure 7, it can be seen that the results are very similar to what was seen in Figure 3, however, this was extracted from a real image obtained from a camera stream. The difference between this step and the one in section 3.1.1 is the equalization method. For Stage 2, Histogram Equalization was replaced with Contrast Limited Adaptive Histogram Equalization (CLAHE). Figure 8 shows the difference between the two methods on the same input image.

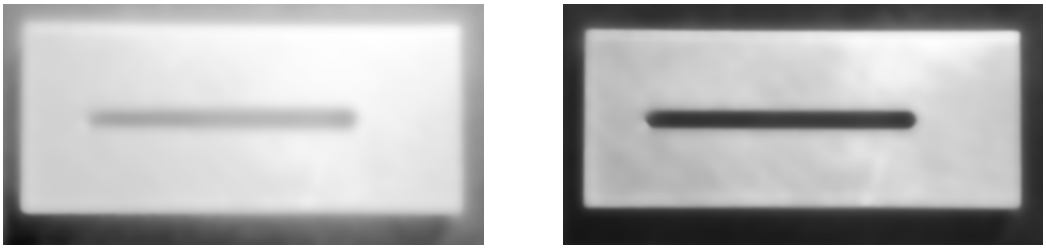

**Figure 8: Histogram Equalization (left) and CLAHE (right)**

From Figure 8, it can be seen that CLAHE is able to create better contrast between the image and its background than Histogram Equalization. According to Verma (July, 2025), CLAHE works by partitioning an image (tiling) into smaller squares and performing histogram equalization on each square. This results in a localized equalization, which allows for better

contrast correction on a local scale. In contrast, histogram equalization is applied globally to the image, disregarding local contrast differences, and causing a less contrasted image as is seen in Figure 8.

### 3.2.2 Geometric Properties

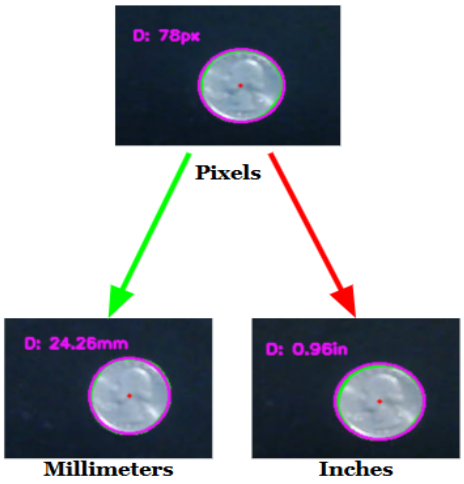


**Figure 9: Unit of Measurement Configuration using a Quarter**

Obtaining the geometric properties of an object is exactly the same as what is seen in section 3.1.2, with an additional step to configure the units of measurement for the process. In order to perform this configuration, an object of known size is required. In my approach I utilized a quarter, which has a known diameter of 24.26mm or 0.955in. The configuration process begins by first extracting the geometric properties of the known object in units of pixels. From Figure 9 it can be seen that the diameter of the quarter was measured to be 78 pixels. The scaling factor to transform pixels-to-inches or pixels-to-millimeters is then determined by dividing inches by the pixel count or the millimeters by pixel count. With the scaling factor determined, the pixel count values of any object are scaled and transformed to the chosen unit of measurement. From Figure 9, it can be seen that the scalings for both inches and millimeters were able to directly scale the pixel count to both units correctly.

### 3.2.3 Offset Measurements

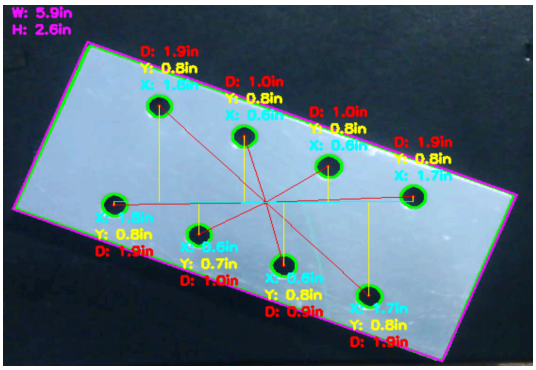


**Figure 10: Offset Measurement Assuming No Object Rotation**

Lastly, the process of determining offset measurements is exactly the same as what was seen in section 3.1.3. However, if we apply the implementation from that section on a real object that has been rotated, such as the one in Figure 10, it can be seen that the vector lengths of each offset vector are different. In reality, we want a result that aligns the offset vectors with the unit vectors of the object frame. For the object in Figure 10, this would result in all the offset vectors having the exact same y-offset vectors, and pairs of the same x-offset vectors depending on the distance of the hole from the centroid of the object. The reason that this occurs is because the offset measurements are being performed in the image frame, and are only represented by the differences between centroids. It does not take into account the rotation of the object, and therefore results in rotational variance between offset vectors.

### 3.2.4 Rotational Invariance

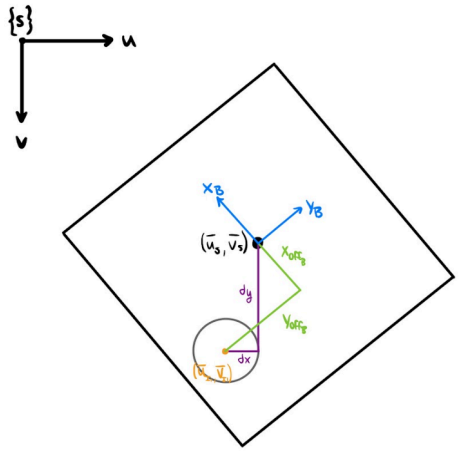


**Figure 11: Problem Definition with Image Frame (u,v) and Object Frame ($x_B$, $y_B$)**

To achieve rotational invariance, we must first define the problem. Firstly, we define the Image Frame (u,v) to be the coordinate system of the image. Secondly, we define the Object Frame ($x_B$, $y_B$) to be the coordinate system of the object. These coordinate systems can be observed in Figure 11. With this defined, we can define the object centroid to be ($\bar{u}_s$,$\bar{v}_s$) and the internal feature centroid to be ($\bar{u}_I$,$\bar{v}_I$). From this it can be seen that **dx** and **dy** are defined in the image frame, and are simply the difference between the object centroid and internal feature centroid. We are, however, interested in obtaining the offset vectors represented in the object frame, shown in Figure 11 as the green lines. We can also see that the object is rotated, and its angle of rotation can be determined by comparing the image frame and object frame. With all of this defined we can solve the problem, which will be done below.

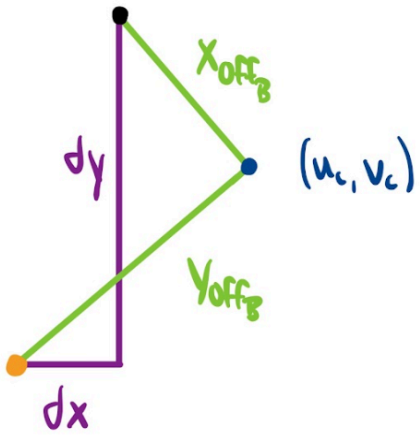


**Figure 12: Outline of Transforming a Point from the Body Frame to the Image Frame**

$$dx_{offB} = dx\cos\emptyset + dy\sin\emptyset \quad \textbf{(5)}$$

$$u_c = \bar{x}_r + dx_{offB}\cos\emptyset \quad \textbf{(6)}$$

$$y_c = \bar{y}_r + dx_{offB}\sin\emptyset \quad \textbf{(7)}$$

Firstly, it can be seen that $x_{offB}$ and $y_{offB}$ are simply a project of the distance from the object centroid to the internal feature centroid onto the object frame. For the purpose of this problem, we are interested in determining only the $x_{offB}$-offset vector distance ($dx_{offB}$) as this will allow to fully define point ($u_c$,$v_c$), which can then be used to plot the rotated offset vectors in the image frame. Equation (5) allows us to solve for the vector distance, which is then used in equation (6) and (7) to determine $u_c$ and $v_c$. With this point computed, we can plot of the offset lines similarly to how was done in section 3.1.3

# 4 Results and Analysis

## 4.1 Test Image Results

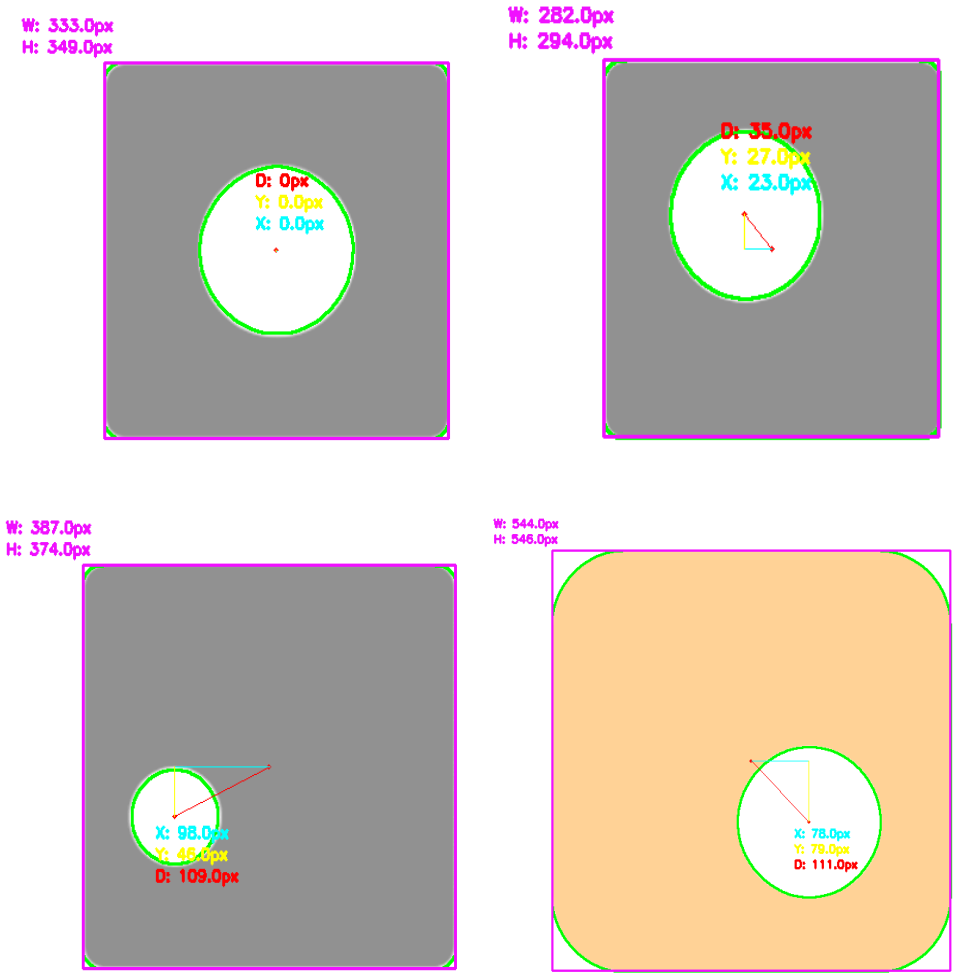


**Figure 13: Offset Measurements of Simple Test Images**

The following results for the simple test images seen in Figure 13 showcase that the software can easily determine all the widths and heights in pixel values, as well as the offset measurements between the object centroids and their respective internal features. Additionally, it was able to handle objects of varying sizes, and colors, and correctly approximated the objects of interest as non-circular. In addition, the top-left image in Figure 13 shows that when the internal feature is perfectly concentric, there is zero offset, which is a correct result.

## 4.2 Simple Live Results

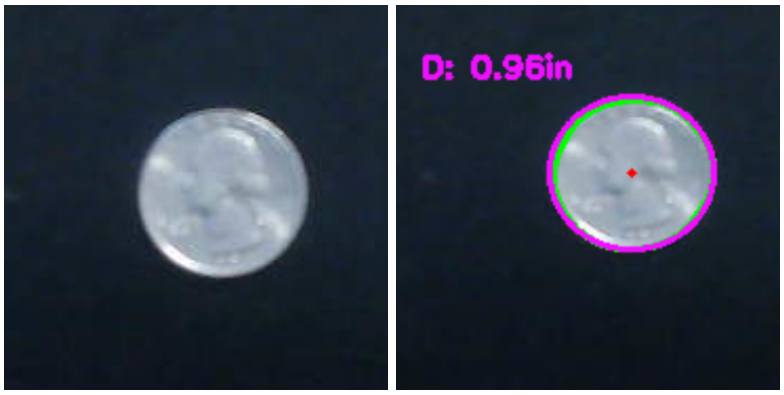


**Figure 14: Geometric Property and Unit of Measurement Test**

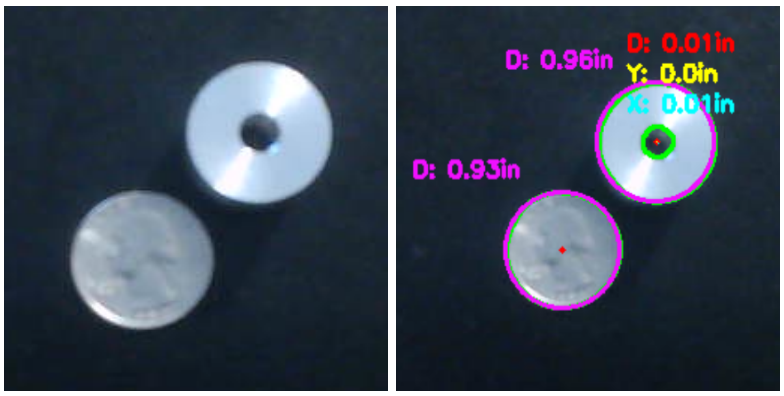


**Figure 15: Multiple Objects with Differing Numbers of Internal Features Test**

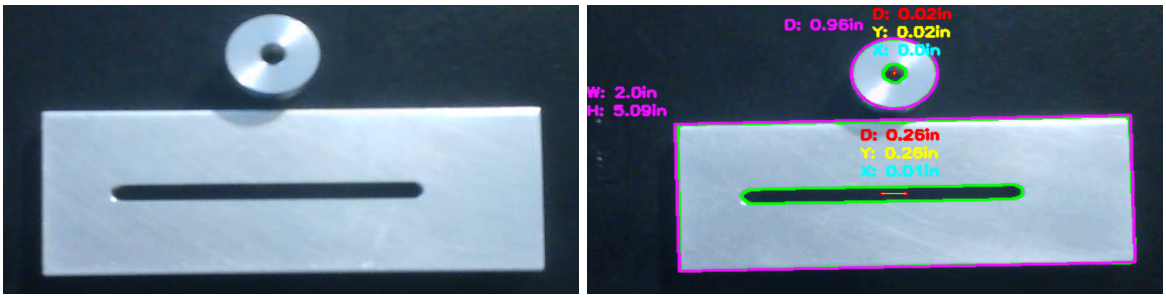


**Figure 16: Multiple Objects with Same Number of Internal Features Test**

Starting with Figure 14, it can be seen that the program was able to correctly determine that the quarter is circular, and was able to correctly identify its diameter in inches. Figure 15 shows that the program is able to handle multiple objects with different numbers of internal

features. In addition to this it is able to determine that the hole on the top right object is concentric with the centroid of that object. A bad result from this figure is that it inaccurately measures the diameter of the quarter, and is likely due to things like poor camera calibration or setup, bad lighting conditions or simply a noisy video stream. Nonetheless, the overall result is good. Lastly Figure 16 showcases that the program is able to handle multiple objects each with their own respective internal features. This result showcases that the program is able to keep track of contour hierarchy and correctly identifies offset measurements, despite there being some noise in the measurements. Additionally, the measured width and height of the rectangular object is correct, as the true dimensions are 2.00in for the width and 5.10in for the height.

### 4.3 Complex Live Results

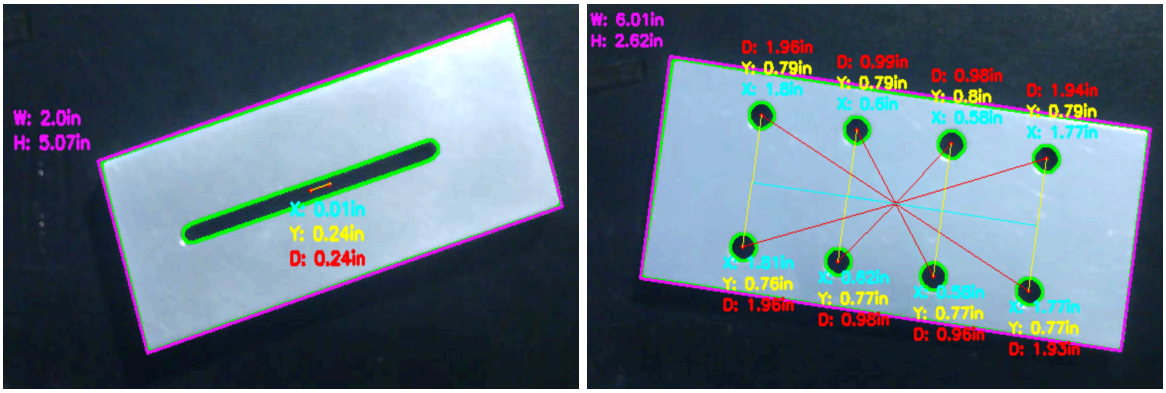


**Figure 17: Rotated Objects with One Internal Feature (left) and Multiple Internal Features (right)**

From Figure 17 it can be seen that with the modifications made in section 3.2.4, the program is now able to handle rotated objects with the offset measurements being rotationally invariant. The left image showcases that the offset measurement of the rotated object with one internal feature is consistent with the “linear” alignment seen in Figure 16. Additionally, the right image showcases that the program is determining the offset measurements for all internal features of the object, and the offset vectors are invariant to rotation.

# 5 Discussion

Overall, the results seen in section 4 showed that the program was very capable of measuring offsets and geometric properties, as well as handling object rotations, varying object sizes, and multiple objects as well as internal features. Nonetheless, there exists room for improvement in the current implementation. Firstly, the measurements were not very stable, and would often vary throughout the video. The most evident case of this is seen in Figure 15 which showed that the measurement of the quarter had changed, as well as in Figure 17, which should have had all the measurements of the parallel internal feature be the same. The most likely reasons for this discrepancy can be attributed to things like lighting variations which could affect edge detection or camera misalignment which could introduce a skew to the image. Additionally, during testing it was found that objects with highly reflective surfaces had poor edge detection as the lighting would break edge cohesion. Furthermore, there was a lack of images with differing colors and texturing in these tests, which makes it uncertain as to whether this program would perform well on those objects.

In order to improve these results, a few things can be done. Firstly, camera calibration can be introduced to detect lens distortion and ultimately correct for it. This would ensure that the image being processed has a linear 2D projection which is necessary for measurement. Furthermore, a proper testbed with a fixed camera pose and control lighting would ensure that the conditions under which the object is measured is controlled. This would allow for objects with highly reflective surfaces to be better analyzed. It would also be important to test objects with different colors and texture to analyze the performance on. In addition to these changes, measurement stabilization can be addressed through a Kalman filter, which could in theory stabilize the detected measurements based on results from previous frames. Lastly, an

improvement which can be added is to add an automatic unit of measurement configuration so that the scaling remains consistent, and is able to update with any change in camera height.

# 6 Conclusion

In conclusion the results of testing the program have shown that it can reliably determine offset measurements, geometric properties, and handle numerous challenges like object rotation and multiple internal features. Despite the results being promising, there still exist challenges with reliable and consistent measurements, requiring further inspection in stabilization methods. Furthermore it is important to measure the measurement accuracy of the tool. Nonetheless, the results are promising and should be further investigated.

# 7 References Cited

Paris, S., Kornprobst, P., Tumblin, J., & Durand, F. (2008). Bilateral filtering: Theory and applications - people | MIT CSAIL. https://people.csail.mit.edu/sparis/publi/2009/fntcgv/Paris_09_Bilateral_filtering.pdf

Image Thresholding. OpenCV. (n.d.). https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

Verma, A. (2025, July 12). Clahe histogram equalization - opencv. GeeksforGeeks. https://www.geeksforgeeks.org/python/clahe-histogram-eqalization-opencv/